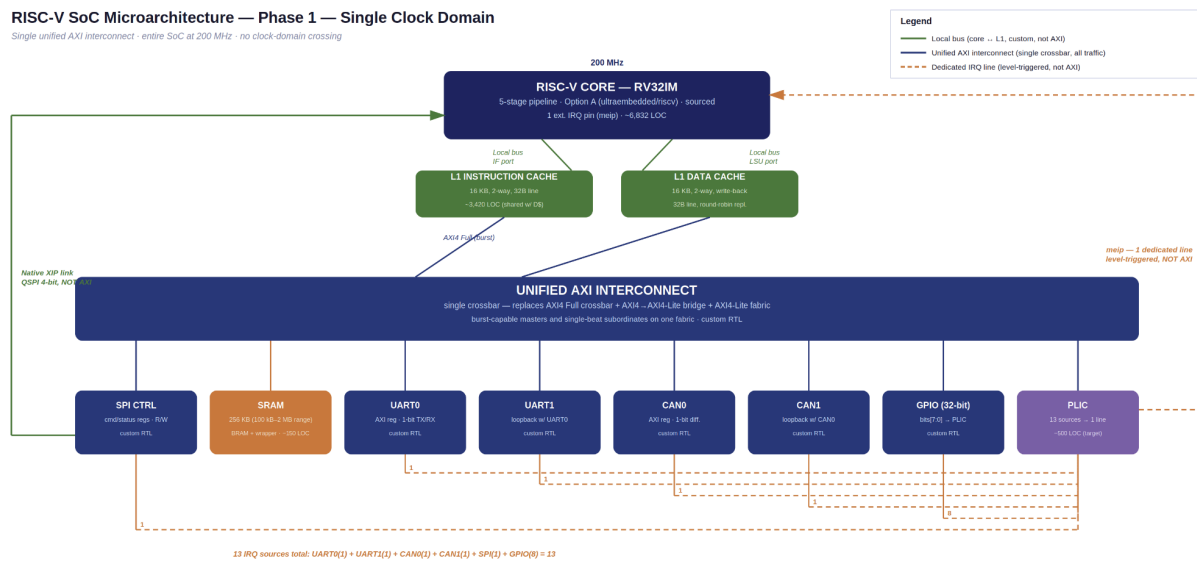


RISC-V SoC Microarchitecture Specification

Status: Core, sourcing split, interconnect topology, interrupt source count, and Phase 1 clocking are locked. Phase 2 clocking is scoped with one open parameter (see § 5).

Revision: v1

2. Phase 1 — Single Clock Domain



Clock: single domain, entire SoC @ 200 MHz. No clock-domain crossing anywhere in this phase.
PLIC priority: fixed, in-order by source ID (see microarchitecture doc § Interrupt Architecture for the ID table).

Figure 1: Phase 1 microarchitecture — single clock domain, unified AXI interconnect, 13-source interrupt architecture.

2.1 System Overview

The confirmed block set is: RISC-V core (RV32IM), L1 instruction/data cache, a single unified AXI interconnect, CAN0/CAN1, UART0/UART1, GPIO, SRAM, an interrupt controller (PLIC-style), and an SPI boot controller with read/write flash access. The RISC-V core and its L1 cache are sourced from an existing open-source implementation (§ 2.2); every other block is custom RTL, written for this project rather than pulled from an external repository.

Clock: 200 MHz, single domain, entire SoC. No clock-domain crossing exists anywhere in this phase.

Reset: single reset domain, a direct consequence of the single clock domain.

Interconnect topology: the AXI4 Full crossbar, the AXI4-to-AXI4-Lite bridge, and the AXI4-Lite peripheral fabric are collapsed into a single unified AXI interconnect. AXI4-Lite is a functional subset of AXI4; the unified crossbar serves the core's burst-capable master traffic and each peripheral's single-beat register traffic without a separate protocol-translation block. Peripheral subordinate ports remain simple, single-beat interfaces — no peripheral is required to implement burst logic it does not use.

SPI boot controller: supports both read (execute-in-place boot fetch) and write/erase (firmware update storage) operations. The controller exposes two interfaces: a native, non-AXI execute-in-place (XIP) link directly to the core for instruction fetch, and a command/status register interface on the unified AXI interconnect for firmware to issue write/erase commands and poll completion.

2.2 Processor Core, L1 Cache, and AXI Full Interconnect

Source: ultraembedded/riscv, the `top_cache_axi` build variant (Option A), BSD-style license. The core, its L1 instruction cache, and its L1 data cache all come from this single repository. The properties below are taken directly from that source — they are fixed characteristics of the sourced RTL, not design choices open to this project.

Processor Core

- ISA: RV32IM, with the Zicsr extension for control/status register access (interrupt and exception handling).
- Implementation: 5-stage pipeline — Instruction Fetch, Instruction Decode, Execute, Memory Access, Writeback (~6,832 LOC).
- Register file: 32×32 -bit general-purpose registers (x0 hardwired to zero), per the RV32 base ISA.
- Privilege modes: the upstream core supports machine, supervisor, and user modes with basic MMU support. This SoC configuration uses machine mode (M-mode) only — no operating system or virtual memory is required — which is a usage choice, not a hardware limitation of the sourced core.
- Interrupt input: one external, active-high interrupt input pin, per the upstream core's documented interface. In this SoC it is level-triggered (meip), setting the machine external interrupt pending bit.
- Reset vector: configured at SoC integration to resolve into the SPI Boot / XIP region (§ 2.4).
- Bus interfaces: two independent AXI4 Full master ports — one from the instruction cache, one from the data cache — into the unified interconnect; one native, non-AXI execute-in-place (XIP) port to the SPI boot controller.

L1 Cache

- L1 Instruction Cache: 16 KB, 2-way set-associative, 256 sets, 32-byte line size.
- L1 Data Cache: 16 KB, 2-way set-associative, 256 sets, 32-byte line size, write-back, allocate on both read and write.
- Total L1 capacity: 32 KB.
- Replacement policy: round-robin — a pointer that increments and toggles between the two ways on each miss. This is not LRU; the upstream source explicitly documents it as a limited pseudo-random scheme, not a true least-recently-used policy.
- These parameters (way count, set count, line size) are hardcoded in the sourced RTL, not exposed as configurable instantiation parameters — changing them would mean modifying the sourced files directly, not simply passing a different value in.
- Core-to-L1 connection: custom, non-AXI local bus with separate instruction-fetch (IF) and load-store-unit (LSU) ports — direct point-to-point, no burst/handshake overhead.

AXI Full Burst Behavior

Each cache services a miss with a single 8-beat INCR burst on its AXI4 Full master port (burst length encoded as 7, 4 bytes per beat), which is exactly one 32-byte cache line in one transaction. The data cache uses the same 8-beat burst length when writing back a dirty line on eviction. This is why the interconnect's SRAM subordinate must

support AXI4 Full burst transactions, while every other subordinate on the interconnect only ever sees single-beat register traffic (§ 2.3).

2.3 System Interconnect

A single unified AXI interconnect is the only on-chip fabric between the core-side masters and every subordinate in the system. It replaces what would otherwise be a three-block chain — an AXI4 Full crossbar, an AXI4-to-AXI4-Lite protocol bridge, and a separate AXI4-Lite peripheral fabric — with one crossbar.

Parameter	Value
Address width	32-bit
Data width	32-bit
Masters	2 — L1 instruction cache, L1 data cache (both full AXI4, burst-capable)
Subordinates	8 — SRAM, UART0, UART1, CAN0, CAN1, GPIO, interrupt controller, SPI boot controller (cmd/status)
Transaction types	SRAM accepts burst transactions (cache line fills). All other subordinates are single-beat only — a functional subset of AXI4, equivalent to AXI4-Lite behavior.
Master arbitration	Fixed priority, instruction cache over data cache (an instruction-cache miss stalls the full pipeline; a data-cache miss stalls only the memory stage)
Outstanding transactions	1 per master (non-pipelined arbitration) in Phase 1

2.4 System Memory Map

The address map below is defined at SoC integration. Each peripheral is assigned a page-aligned 4 KB register window regardless of actual register count, per standard practice for address decode simplicity and future expansion.

Region	Address Range	Size	Notes
SPI Boot / XIP	0x0000_0000 – 0x0FFF_FFFF	256 MB window	Execute-in-place from QSPI flash; core reset vector resolves here
SRAM	0x1000_0000 – 0x101F_FFFF	2 MB window (256 KB populated)	AXI4 Full subordinate, burst-capable
Reserved	0x2000_0000 – 0x7FFF_FFFF	—	Reserved for future expansion
UART0	0x8000_0000 – 0x8000_0FFF	4 KB	Register window
UART1	0x8000_1000 – 0x8000_1FFF	4 KB	Register window
CAN0	0x8000_2000 – 0x8000_2FFF	4 KB	Register window
CAN1	0x8000_3000 – 0x8000_3FFF	4 KB	Register window
GPIO	0x8000_4000 – 0x8000_4FFF	4 KB	Register window
Interrupt Controller (PLIC)	0x8000_5000 – 0x8000_5FFF	4 KB	Register window
SPI Boot Controller	0x8000_6000 – 0x8000_6FFF	4 KB	Command/status registers only — XIP path is not memory-mapped through this window
Reserved	0x8000_7000 – 0xFFFF_FFFF	—	Reserved for future expansion

2.5 Peripheral Subsystem

2.5.1 UART0 / UART1

Two independent UART instances, each with its own register window on the unified interconnect. Beyond their normal role as a serial console, the two instances are cross-wired on-chip (UART0 TX ↔ UART1 RX, UART1 TX ↔ UART0 RX) so firmware can validate baud rate, parity, and FIFO handling by talking to itself, without a second board or a USB-serial cable.

- Interface: register access on the unified interconnect, 4 KB window each (§ 2.4).
- Register blocks: Control, Status, TX Data, RX Data, Baud/Clock Divisor, Interrupt Enable.
- Serial: 1-bit TX/RX per instance.

2.5.2 CAN0 / CAN1

Two independent CAN instances. In normal operation each participates in a shared automotive/industrial control bus as one node among several. Like the UARTs, the pair is also cross-wired on-chip so collision handling and error-frame behavior can be validated on the FPGA itself, without a second physical board or external CAN test equipment.

- Interface: register access on the unified interconnect, 4 KB window each (§ 2.4).
- Register blocks: Control, Status, TX Buffer, RX Buffer/FIFO, Bit-Timing Configuration, Interrupt Enable, Error Counter.
- Serial: 1-bit differential per instance.

2.5.3 GPIO

A single 32-bit parallel I/O block for general digital input/output — sensors, actuators, status and control lines. Direction is set per pin via a masking register. The lower 8 bits are individually wired to the interrupt controller rather than sharing a combined line, since each can independently be configured as rising-edge, falling-edge, or state-change triggered.

- Interface: register access on the unified interconnect, 4 KB window (§ 2.4).
- Register blocks: Data (input/output), Direction (mask), Interrupt Enable, Interrupt Type/Polarity (per bit), Interrupt Status.

2.5.4 Interrupt Controller (PLIC-style)

Aggregates every interrupt source in the system — one combined line per device plus the 8 individual GPIO bit lines — and presents a single line to the core. Full source list, priority order, and signal path are detailed in § 2.6.

- Interface: register access on the unified interconnect, 4 KB window (§ 2.4).
- Register blocks: Source Priority, Interrupt Enable (per source), Interrupt Pending, Claim/Complete.

2.5.5 SPI Boot Controller

Boots the chip from external Quad-SPI (QSPI) flash and also supports writing firmware updates back to that same flash in the field — this block is both the boot path and the only persistent storage device in the design. It exposes two separate interfaces for these two roles: a native, non-AXI execute-in-place (XIP) port straight to the core for instruction fetch, and a command/status register interface on the unified interconnect for firmware to issue write/erase commands and poll completion.

- Register blocks (command/status side): Command, Status, Address, Data/FIFO, Interrupt Enable.
- Physical interface: Quad-SPI (QSPI), 4-bit wide data bus.
- Operating frequency: 25 MHz (200 MHz Phase 1 clock ÷ 8) — re-derived from the original 25 MHz target, since the base clock changed from the 50 MHz previously assumed.

2.6 Interrupt Architecture

Source count: 13 — one combined interrupt per device (CAN0, CAN1, UART0, UART1, SPI Boot Controller) plus 8 individually wired GPIO bit interrupts. On a combined per-device interrupt, firmware reads that device's own status register to determine the specific cause (e.g. RX-ready vs. error).

Priority: fixed, in-order by source ID — lower ID services first. Non-preemptive: a source being serviced is not interrupted by a higher-priority source until the current claim/complete cycle finishes.

ID	Source	Priority	Description
0	CAN0_IRQ	1 (highest)	CAN0 combined interrupt; cause read from CAN0 status register
1	CAN1_IRQ	2	CAN1 combined interrupt; cause read from CAN1 status register
2	UART0_IRQ	3	UART0 combined interrupt; cause read from UART0 status register
3	UART1_IRQ	4	UART1 combined interrupt; cause read from UART1 status register
4	SPI_IRQ	5	SPI boot controller combined interrupt (write/erase complete, error); cause read from SPI status register
5	GPIO_IRQ[0]	6	GPIO bit 0 edge interrupt, polarity programmable per bit
6	GPIO_IRQ[1]	7	GPIO bit 1 edge interrupt
7	GPIO_IRQ[2]	8	GPIO bit 2 edge interrupt
8	GPIO_IRQ[3]	9	GPIO bit 3 edge interrupt
9	GPIO_IRQ[4]	10	GPIO bit 4 edge interrupt
10	GPIO_IRQ[5]	11	GPIO bit 5 edge interrupt
11	GPIO_IRQ[6]	12	GPIO bit 6 edge interrupt
12	GPIO_IRQ[7]	13 (lowest)	GPIO bit 7 edge interrupt

Signal path to core: a single dedicated, level-triggered line (meip) from the interrupt controller directly into the core's external interrupt pin. This line is not part of the unified AXI interconnect. The interconnect carries only the interrupt controller's configuration path — priority, enable/mask, and claim/complete registers — not the interrupt signal itself.

2.7 Boot and Reset Sequence

- On reset de-assertion, the core begins fetch at the reset vector, which resolves into the SPI Boot / XIP region (§ 2.4).
- Initial instructions execute directly from QSPI flash over the native XIP link; no bootloader copy-to-SRAM step is required before first fetch.
- Firmware may subsequently copy time-critical code or data into SRAM for lower-latency execution, and may issue write/erase commands through the SPI boot controller's command/status register interface (§ 2.5.5) to update flash contents in the field.
- All peripherals, the interrupt controller, and the unified interconnect are held in reset until the core's reset is released, consistent with the single reset domain defined in § 2.1.